

Agenda

- Orquestación de Casos de Uso
- Capa de Dominio
- Capa de Repositorios
- Capa de Services
- Capa de Controladores
- Manejo de errores y Validaciones



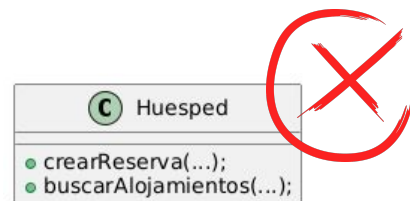


Orquestación de Casos de Uso

Orquestación de Casos de Uso

En el diseño de software, especialmente bajo el paradigma orientado a objetos, es común caer en una ***confusión conceptual***:

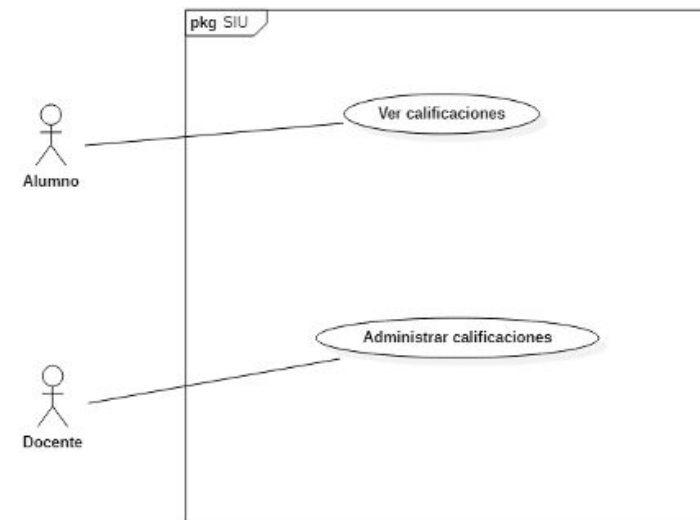
Asignar responsabilidades de casos de uso directamente a clases que representan actores del Sistema.



Orquestación de Casos de Uso

Por ejemplo, si consideramos el Sistema de Gestión Educativa SIU Guaraní, el diagrama de Casos de Uso expuesto y los siguientes requerimientos:

- El Sistema debe permitir que los alumnos visualicen sus calificaciones
- El Sistema debe permitir la administración (alta, baja y modificación) de calificaciones por parte de los docentes





Orquestación de Casos de Uso

En un primer intento, quizás haríamos algo como:

```
class Alumno {  
    void verCalificaciones() { ... }  
}
```

```
class Docente {  
    void administrarCalificaciones() { ... }  
}
```

Pero... ¿qué hacen esos métodos?





Orquestación de Casos de Uso

Este diseño tiene varios problemas de diseño ya que:

- Mezcla el modelo de dominio (*Alumno, Docente*) con la orquestación de casos de uso (*Ver Calificaciones, Administrar Calificaciones*).
- Confunde el propósito de cada clase, asignándole lógica que no le corresponde.
- Impide reutilizar la lógica de negocio en otros contextos (API REST, CLI, etc.).





Orquestación de Casos de Uso

Lo que significa que:

- De estos requerimientos no se desprende la necesidad de que deba haber una clase Alumno que "*vea sus calificaciones*", ni una clase Docente que "*administre calificaciones*".





Orquestación de Casos de Uso

Entonces:

El foco no debe estar en quién lo hace, sino en qué se hace y cómo se orquesta esa lógica de negocio.





Orquestación de Casos de Uso

La solución es separar claramente la lógica de orquestación (casos de uso) de los modelos de dominio y las políticas de acceso.





Capa de Dominio



Capa de Dominio

- La capa de Dominio contiene la lógica de negocio pura: ***reglas, validaciones, entidades y objetos de valor.***
- Es el corazón del Sistema y debe estar diseñada e implementada de forma independiente de la infraestructura o de frameworks.



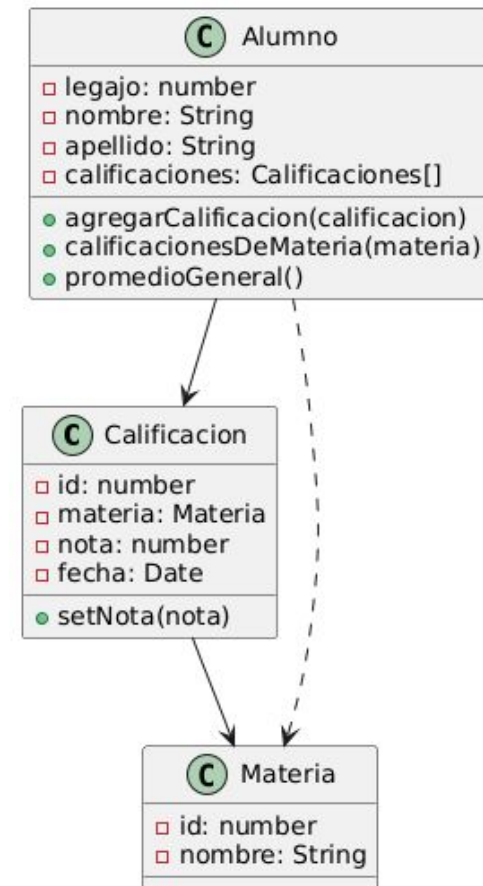


Capa de Dominio

¡Es la Capa que venimos trabajando desde las demás asignaturas previas!

Capa de Dominio

Si nos contextualizamos en el ejemplo del SIU, podríamos plantear el siguiente diagrama de clases...





```
export class Materia {  
  constructor({ id, nombre }) {  
    if (!id || !nombre) throw new Error("Materia debe tener  
id y nombre.");  
  
    this.id = id;  
    this.nombre = nombre;  
  }  
}
```



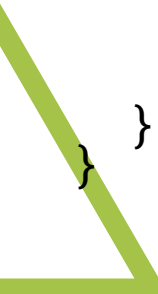


```
export class Calificacion {
  constructor({ id, materia, nota, fecha }) {
    if (!materia || typeof materia !== 'object') {
      throw new Error("Materia inválida.");
    }

    if (nota < 0 || nota > 10) {
      throw new Error("La nota debe estar entre 0 y 10.");
    }

    this.id = id;
    this.materia = materia;
    this.nota = nota;
    this.fecha = fecha;
  }

  setNota(nuevaNota) {
    if (nuevaNota < 0 || nuevaNota > 10) {
      throw new Error("La nota debe estar entre 0 y 10.");
    }
    this.nota = nuevaNota;
  }
}
```



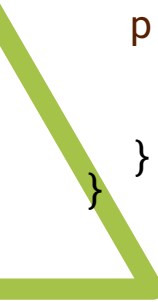


```
export class Alumno {
  constructor({ legajo, nombre, apellido }) {
    if (!legajo || !nombre || !apellido) {
      throw new Error("Faltan datos obligatorios del alumno.");
    }
    this.legajo = legajo;
    this.nombre = nombre;
    this.apellido = apellido;
    this.calificaciones = [];
  }

  agregarCalificacion(calificacion) {
    if (!(calificacion instanceof Calificacion)) {
      throw new Error("Instancia de Calificacion inválida.");
    }
    this.calificaciones.push(calificacion);
  }

  calificacionesDeMateria(materia) {
    return this.calificaciones.filter(c => c.materia.id === materia.id);
  }

  promedioGeneral() {
    if (this.calificaciones.length === 0) return null;
    const suma = this.calificaciones.reduce((acc, c) => acc + c.nota, 0);
    return suma / this.calificaciones.length;
  }
}
```





Capa de Dominio

¿Qué estamos viendo acá?

- “Calificación” es una entidad rica en lógica: sabe validarse a sí misma (setNota). Esto también vale para “Materia” y “Alumno”
- La lógica de negocio vive en las entidades y en los casos de uso, no en los actores.
- No hay mención a los CU en Alumno, ni aparece Docente, ni se hace referencia a los permisos, ni interfaces. Es puro core del sistema.





Capa de Dominio

En este ejemplo, y en general para esta capa:

- Existe una separación clara de responsabilidades entre las distintas clases.
- Las entidades son autocontenidas y validadas.
- Se evita mezclar permisos, interfaces o actores externos.
- El control de acceso se encuentra fuera de esta capa. En capas superiores se define si el que está llamando es un Docente, si tiene permiso, si viene desde una interfaz web o una API REST.





Capa de Dominio

Ventajas

- Fácil de testear (no depende de base de datos ni frameworks).
- Reutilizable y flexible ante cambios en otras capas.

Code Smells Evitados

- Anemic Domain Model: se evita centralizando la lógica en las entidades, no en los servicios.
- Duplicación de reglas de negocio: al estar centralizadas en el dominio, se evita repetirlas en otros lugares.





Capa de Dominio

Principios SOLID aplicados

- O (Open - Close): las entidades pueden extenderse sin modificar su código base.
- L (Liskov Substitution): objetos del dominio pueden reemplazarse sin alterar el funcionamiento.





Capa de Repositorios



Capa de Repositorios

- Encapsula el acceso al medio persistente, que puede ser: memoria, archivos, bases de datos, etc.
- Traduce las entidades del dominio a un formato persistente y viceversa.





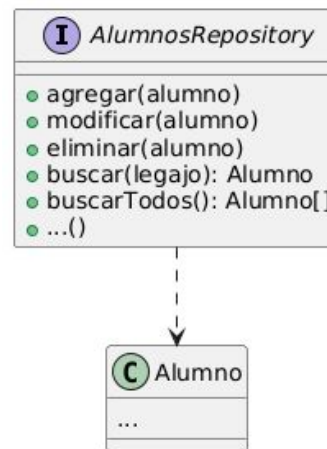
Capa de Repositorios

- Propone que en la capa de persistencia existan objetos “*Repository*” .
- Cada Repositorio debería poder:
 - agregar(unObjeto)
 - modificar(unObjeto)
 - eliminar(unObjeto)
 - buscar
 - buscarTodos
 - ...



Capa de Repositorios

Situados en nuestro ejemplo del SIU...





Si pensamos en un repositorio que guarde sus elementos en memoria:

```
export class AlumnoRepository {  
  constructor() {  
    this._alumnos = new Map(); // simulamos persistencia  
  }  
  
  buscar(legajo) {  
    return this._alumnos.get(legajo) || null;  
  }  
  
  agregar(alumno) {  
    this._alumnos.set(alumno.legajo, alumno);  
  }  
}
```





Capa de Repositorios

Ventajas

- Permite cambiar la tecnología de persistencia sin afectar el dominio.
- Facilita el testing mediante mocks o implementaciones in-memory.





Capa de Repositorios

Principios SOLID aplicados

- I (Interface Segregation): una interfaz de repositorio expone solo lo necesario.
- D (Dependency Inversion): los services usan las interfaces, no dependen de implementaciones concretas de repositorios.





Capa de Services



Capa de Services

- La capa de Services orquesta la ejecución de una operación de negocio.
- Llama a los objetos de dominio, repositorios y otros objetos de utilidad.





Capa de Services

- ***Separa el "cómo se hace algo" (dominio) del "cuándo y en qué orden" se hace (servicio)***
- ***Focaliza las reglas del flujo, no las reglas de negocio.***





Capa de Services

Retomando el ejemplo del SIU y el CU “*Registrar una calificación de un alumno*”...

Se requiere

- Validar que el alumno exista.
- Validar que la materia exista.
- Crear la calificación.
- Agregarla al alumno.
- Persistir (guardar) la calificación



Capa de Services

```
export class AlumnoService {  
  constructor({ alumnoRepo, materiaRepo }) {  
    this.alumnoRepo = alumnoRepo; // inyección de dependencias  
    this.materiaRepo = materiaRepo;  
  }  
  ...  
}
```




```
export class AlumnoService {

  registrarCalificacion({ legajo, materiaId, nota, fecha }) {
    const alumno = this.alumnoRepo.buscar(legajo);
    if (!alumno) throw new Error("Alumno no encontrado.");

    const materia = this.materiaRepo.buscarPorId(materiaId);
    if (!materia) throw new Error("Materia no encontrada.");

    const calificacion = new Calificacion({id: crypto.randomUUID(), materia, nota, fecha: fecha || new Date()});

    alumno.agregarCalificacion(calificacion);
    this.alumnoRepo.guardar(alumno);

    return calificacion;
  }}

```





Capa de Services

Ahora consideremos un CU de “*Visualizar el promedio general de un Alumno*”...



```
export class AlumnoService {  
  obtenerPromedioDeAlumno(legajo) {  
    const alumno = this.alumnoRepo.buscar(legajo);  
    if (!alumno) throw new Error("Alumno no encontrado.");  
    return alumno.promedioGeneral();  
  }  
}
```





Capa de Controladores



Capa de Controladores

- Esta capa se encuentra más “arriba” de la capa de Dominio/Negocio, más precisamente por encima de la capa de Services y por debajo de la capa de Presentación.
- Esta capa recibe las solicitudes externas (web, API, etc.), las traduce al lenguaje de domino, las delega en las capas inferiores para que las resuelvan, y devuelve la respuesta esperada.





Capa de Controladores

- Esta capa expone los **endpoints** del Sistema, estableciendo claramente sus firmas.
- Al recibir una petición, traduce los inputs al lenguaje del dominio (si todavía no está hecho), y luego delega a la capa de Services la responsabilidad.
- Espera que la capa de Services le devuelva el/los objeto/s esperado/s, para generar una respuesta y devolverla.





Capa de Controladores

- También es responsable de manejar excepciones, códigos HTTP, validaciones superficiales (tipo de datos, campos faltantes).
- Nunca debe haber lógica de negocio real acá.
- Un “buen” controlador delegador y explícito.





Capa de Controladores

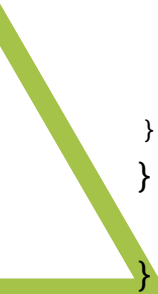
Retomando nuestro ejemplo del SIU...





```
class AlumnoController {
  constructor(alumnoService) {
    this.alumnoService = alumnoService;
  }

  registrarCalificacion(req, res) {
    const { legajo } = req.params;
    const { materiaId, nota } = req.body;
    if (!materiaId || typeof nota !== 'number') {
      return res.status(400).json({ error: 'materiaId y nota son requeridos.' });
    }
    try {
      const calificacion = this.alumnoService.registrarCalificacion({legajo, materiaId, nota});
      return res.status(201).json({
        id: calificacion.id,
        materia: calificacion.materia.nombre,
        nota: calificacion.nota,
        fecha: calificacion.fecha
      });
    } catch (error) {
      return res.status(400).json({ error: error.message });
    }
  }
}
```





```
class AlumnoController {  
  ...  
  
  obtenerPromedio(req, res) {  
    const { legajo } = req.params;  
  
    try {  
      const promedio = this.alumnoService.promedioDeAlumno(legajo);  
      return res.json({ promedio });  
    } catch (error) {  
      return res.status(404).json({ error: error.message });  
    }  
  }  
}
```





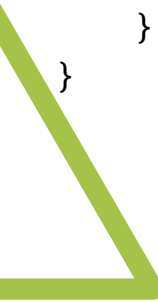
```
class AlumnoController {
  ...

  configurarRutas() {
    const router = express.Router();

    router.post('/:alumnoId/calificaciones', this.registrarCalificacion.bind(this));
    router.get('/:alumnoId/promedio', this.obtenerPromedio.bind(this));

    return router;
  }

  static crearAlumnoController({ alumnoService }) {
    const controller = new AlumnoController(alumnoService);
    return controller.configurarRutas();
  }
}
```



>> main.js

```
const app = express();  
app.use(bodyParser.json());
```

```
//...
```

```
app.use("/alumnos", AlumnoController.crearAlumnoController({alumnoService}));
```

```
//...
```



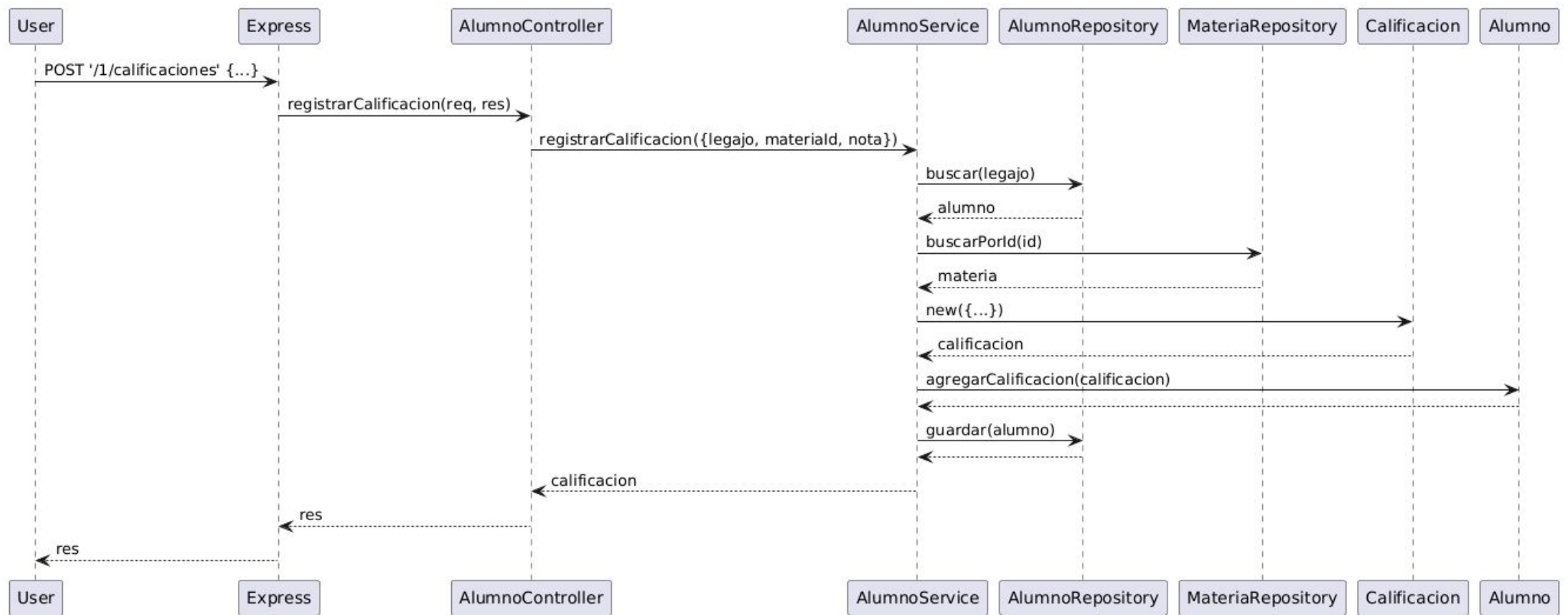
Capa de Controladores

Ventajas

- Cambios en el frontend no afectan la lógica interna.
- Mantenible y simple, se encarga solo de parsear y responder.



Ejemplo flujo completo





Manejo de Errores y Validaciones



Validaciones

- La validación es una defensa contra datos inesperados o maliciosos que pueden comprometer el sistema.
- Se puede dar en distintos niveles:
 - **Cliente/UI**: validaciones rápidas para mejorar UX.
 - **Controladores**: validaciones de estructura y presencia de campos.
 - **Capa de Servicios/Dominio**: validaciones de reglas de negocio y consistencia interna.
 - **Persistencia**: validaciones de integridad de datos (ej: claves únicas, tipos de datos, etc.).



Validaciones

<i>Nivel</i>	<i>Ejemplo</i>	<i>Justificación</i>
UI/Controller	<code>if (!email.includes("@"))</code>	Validación básica
Servicio	<code>if (!alumno.puedeSerCalificado(materia))</code>	Regla de negocio compleja
Dominio	<code>calificacion.setNota chequea nota >= 1 && nota <= 10</code>	Inmutabilidad y consistencia
Repositorios	Restricción UNIQUE en legajo	Capa final de defensa



Errores y Excepciones

Los errores bien manejados permiten:

- Separar responsabilidades entre capas.
- Mejorar el debugging.
- Dar respuestas útiles al usuario/API client.
- Loguear adecuadamente problemas reales.





Errores y Excepciones

Es recomendable declarar los Errores que queremos utilizar en nuestro proyecto de forma personalizada, para que al utilizarlos las excepciones puedan ser mejor tratadas.





Declaración de Errores

```
export class ValidationError extends Error {  
  constructor(message) {  
    super(message);  
    this.name = 'ValidationError';  
  }  
}
```

```
export class NotFoundError extends Error {  
  constructor(message) {  
    super(message);  
    this.name = 'NotFoundError';  
  }  
}
```





Manejo de Excepciones

Un enfoque común y limpio para el manejo de excepciones, inspirado en `@ControllerAdvice` de Spring Boot (framework de Java), es tener un middleware global de errores en Express.





```
export function errorHandler(err, req, res, next) {  
  console.error(err);  
  
  if (err.name === 'ValidationError') {  
    return res.status(400).json({ error: err.message });  
  }  
  
  if (err.name === 'NotFoundError') {  
    return res.status(404).json({ error: err.message });  
  }  
  
  // error desconocido  
  return res.status(500).json({ error: 'Error interno del servidor' });  
}
```





```
>> main.js
```

```
//..
```

```
import { errorHandler } from './middlewares/errorHandler.js';
```

```
// otros middlewares y rutas
```

```
app.use(errorHandler);
```





¡Gracias!